NIA AI Talking Avatar — Python Backend Blueprint

1. Project Overview

Build the Python backend for the Noida International Airport (NIA) AI Talking Avatar.

The backend will initially run locally in Docker and connect to an Ollama model. It will manage airport knowledge, answer user questions, and support two knowledge approaches:

- **OKF:** Loads approved Markdown documents and provides the full knowledge context to the language model.
- **RAG:** Retrieves relevant information from documents using hybrid keyword and semantic search, with special handling for tables.

Both approaches must be implemented and evaluated against the same ground-truth questions. We will not assume either approach is better until the results are measured.

React will be developed separately and is outside the scope of this initial Python backend.

2. Main Objectives

1. Create a modular Python backend using FastAPI.
2. Run FastAPI inside a Docker container.
3. Connect to a locally running Ollama model.
4. Keep model, storage, and application settings configurable.
5. Load airport knowledge from approved Markdown files.
6. Implement OKF and RAG as separate strategies.
7. Compare their accuracy and latency using a common evaluation system.
8. Support future migration to AWS or another cloud provider without rewriting the core application.
9. Keep test datasets separate from production knowledge.
10. Prepare the backend for future speech recognition, text-to-speech, and avatar integration.

3. Technology Plan

Component	Initial choice
Programming language	Python
API framework	FastAPI
API server	Uvicorn
Containerization	Docker and Docker Compose

Component	Initial choice
Local language model	Ollama
Configuration	Environment variables and `.env`
Knowledge format	Markdown
Knowledge strategies	OKF and RAG
RAG retrieval	Hybrid lexical and vector search
Initial storage	Local filesystem
Future storage	AWS S3 and CloudFront
Testing	Pytest
Frontend	React, developed separately

Specific vector databases, embedding models, and cloud services can be selected during implementation after requirements and benchmarks are established.

4. Recommended Folder Structure

```
nia-avatar/
│
├── README.md
├── .gitignore
├── .dockerignore
├── Dockerfile
├── compose.yaml
│
└── backend/
    ├── pyproject.toml
    ├── .env.example
    │
    ├── data/
    │   ├── manifest.json
    │   ├── nia_airport/
    │   │   └── approved Markdown documents
    │   ├── ground_truth.json
    │   └── unknown_traps.json
    │
    ├── src/
    │   └── nia_backend/
    │       ├── main.py
    │       │
    │       ├── api/
    │       ├── core/
    │       ├── schemas/
    │       ├── models/
    │       ├── knowledge/
    │       ├── strategies/
    │       ├── retrieval/
    │       ├── services/
    │       ├── storage/
    │       └── evaluation/
    │
    └── tests/
```

Purpose of each folder

`api/` — API endpoints

Contains FastAPI routes for health checks, chat questions, knowledge status, knowledge refresh, and evaluation.

`core/` — Application settings

Contains environment-based configuration, logging, application-wide errors, and shared settings.

`schemas/` — Request and response definitions

Defines the structure of incoming questions, answers, source references, knowledge status, and evaluation results.

`models/` — Language model connections

Contains the common model-provider interface and the Ollama implementation. Future cloud model providers will implement the same interface.

`knowledge/` — Knowledge ingestion

Handles manifest validation, Markdown loading, document parsing, metadata, document versions, and knowledge snapshots.

`strategies/` — OKF and RAG

Contains the common knowledge-strategy interface, the OKF strategy, the RAG strategy, and a possible hybrid strategy.

`retrieval/` — RAG search system

Contains document-aware chunking, table parsing, lexical search, vector search, ranking, and retrieval evaluation support.

`services/` — Application workflows

Coordinates chat requests, evidence validation, answer generation, and knowledge refresh.

`storage/` — Storage abstraction

Provides a common interface for reading knowledge from local files now and cloud storage later.

`evaluation/` — Benchmarking

Runs the ground-truth test suite and measures answer accuracy, retrieval accuracy, table correctness, source attribution, latency, and resource usage.

`tests/` — Automated tests

Tests manifest validation, ingestion, OKF, RAG, chat behavior, security checks, and cloud-adapter compatibility.

5. Knowledge Data Structure

The data directory will contain three separate categories.

`manifest.json`

The manifest defines which Markdown documents are approved for ingestion. Each document entry should support:

- Document identifier
- Relative file path
- Official title
- Category
- Last-updated date
- Ingestion enabled/disabled flag

Only approved Markdown files with ingestion enabled may enter the runtime knowledge system.

`nia_airport/`

Contains the actual airport knowledge documents. Keep Markdown tables, headings, lists, units, and important notes intact.

`ground_truth.json`

Contains evaluation questions, expected answers, and the source information needed to verify correctness. It must not be loaded into the production knowledge context.

`unknown_traps.json`

Contains adversarial questions, false premises, and questions that test whether the assistant invents unsupported information. It must never be ingested as production knowledge.

6. OKF Architecture

The OKF approach will:

1. Read the approved document manifest.
2. Load approved Markdown files.
3. Validate the files and attach source metadata.
4. Preserve complete Markdown documents and their tables.
5. Assemble a knowledge context in memory.
6. Send the context with the user's question to the language model.
7. Validate the generated answer and retain source references.
8. Abstain when the evidence is insufficient.

The implementation must account for context-window limits and token usage. Fast memory access does not guarantee fast end-to-end answers, because the language model still needs to process the supplied context.

7. RAG Architecture

The RAG approach will use the same approved source documents as OKF.

It will:

1. Load and parse approved Markdown documents.
2. Split ordinary prose into document-aware chunks.
3. Preserve headings and source locations.
4. Preserve original tables as canonical evidence.
5. Create searchable table-row records that retain headers, units, and relevant notes.
6. Build a lexical search index.
7. Build a vector search index.
8. Retrieve and combine results from both search methods.
9. Rank relevant evidence and expand table matches where necessary.
10. Pass the selected evidence to the language model.
11. Validate the answer and provide source references.
12. Abstain when the retrieved evidence cannot support a reliable answer.

Initial chunk sizes and retrieval settings should be treated as configurable starting points, then tuned using the benchmark.

8. Shared Knowledge Pipeline

Both strategies must use the same source of truth.

The pipeline will be:

1. Maintain approved Markdown documents and the manifest in GitHub.
2. Validate the manifest and document paths.
3. Publish the approved source files.
4. Initially read files from the local filesystem.
5. Later support publishing to S3 and distribution through CloudFront.
6. Validate the incoming knowledge snapshot.
7. Prepare the OKF context and RAG indexes independently.
8. Activate a complete, validated snapshot.
9. Retain the previous valid snapshot if a refresh fails.

The application must not activate a partially loaded or invalid knowledge snapshot.

The knowledge refresh operation must be protected before the API is exposed outside a trusted development environment.

9. Python API Responsibilities

The initial backend should support these operations:

Endpoint	Responsibility
`GET /api/v1/health`	Report API health and basic configuration
`GET /api/v1/knowledge/status`	Report the active knowledge version and document count
`POST /api/v1/knowledge/refresh`	Validate and activate a new knowledge snapshot
`POST /api/v1/chat`	Answer a question using the selected strategy

The chat request should allow the caller to select `okf` or `rag` so the same API can run comparable tests against both strategies.

Evaluation endpoints can be added when the benchmark runner is implemented.

10. Docker and Local Development

FastAPI will run inside a Docker container.

Initially:

- Docker hosts the Python application.
- Ollama runs on the local host machine.
- The model URL is supplied through an environment variable.
- Knowledge files are mounted into the container.
- Knowledge mounts should be read-only where practical.
- Test datasets are kept outside the runtime knowledge mounts.

The backend must bind to `0.0.0.0` inside the container. The host port will be mapped to the container's API port.

Container networking must be configured so FastAPI can reach Ollama. The correct hostname depends on the operating system and Docker environment.

Do not use development auto-reload settings in production.

11. Future Cloud Migration

The architecture must separate application logic from infrastructure.

Model provider

The chat service will call a common model interface. Ollama is the first implementation. A hosted model provider can be added later without changing the chat workflow.

Knowledge storage

The knowledge loader will use a storage interface. Local filesystem access is the first implementation. An AWS S3 implementation can be added later.

Distribution

CloudFront can distribute approved knowledge files or versioned knowledge snapshots. Cache refresh and versioning must be designed so the application does not mix different document versions.

Deployment

The same Docker image should be deployable to a suitable cloud container service. Cloud-specific credentials and configuration must be provided through secure runtime configuration, not hard-coded into the source.

12. Ground-Truth Evaluation Plan

Both strategies must be tested against the same question set and source version.

The dataset should cover:

- Direct factual questions
- Paraphrased questions
- Questions requiring multiple documents
- Table-cell lookups
- Table comparisons
- Ambiguous questions
- Questions with false premises
- Questions whose answers are absent from the knowledge base

Measure:

- Answer correctness
- Retrieval correctness
- Table row, column, and cell accuracy
- Source attribution accuracy
- Appropriate abstention
- Median and p95 retrieval latency
- Median and p95 end-to-end latency
- Knowledge refresh duration
- Token usage
- RAM and index storage usage

Use the same model, question set, source snapshot, and comparable generation settings wherever possible.

The final decision to use OKF, RAG, or a hybrid approach must be based on measured performance rather than assumptions.

13. Implementation Order

1. Create the Python project and folder structure.
2. Configure Docker and local Ollama connectivity.

3. Implement the health endpoint and model-provider interface.
4. Implement manifest validation and Markdown ingestion.
5. Implement safe knowledge snapshot refresh.
6. Complete the OKF strategy.
7. Implement RAG chunking and table-aware indexing.
8. Add hybrid lexical and vector retrieval.
9. Add evidence validation and source attribution.
10. Build the ground-truth benchmark.
11. Compare OKF and RAG for accuracy, latency, and resource usage.
12. Add the cloud storage adapter.
13. Integrate speech recognition, text-to-speech, and avatar communication after the knowledge pipeline is validated.

14. Definition of Done for the Initial Skeleton

The initial skeleton is ready when:

- FastAPI starts successfully in Docker.
- FastAPI can reach the configured Ollama endpoint.
- Environment settings control the model URL and name.
- The manifest admits only approved Markdown documents.
- The test datasets are excluded from runtime knowledge ingestion.
- Knowledge refresh does not replace a valid snapshot with a failed or partial one.
- OKF and RAG are selectable through a common interface.
- The unimplemented RAG path fails safely instead of returning fabricated evidence.
- Automated tests cover the core ingestion and safety rules.
- The project has a documented path to S3/CloudFront and cloud deployment.

Scope for this stage: create the Python backend skeleton, folder structure, configuration plan, Docker setup, and interfaces. Implement the actual application behavior incrementally after the structure is agreed upon.